

INFO8006 Introduction to Artificial Intelligence

Exercise session 1: Solving problems by searching

RECAP

Search problem

A *search problem* is defined by

- the *states* $s \in \mathcal{S}$ and the *initial state* $s_1 \in \mathcal{S}$;
- the *actions* $\text{actions}(s) \subseteq \mathcal{A}$ available in state s ;
- the *transition model* $s' = \text{result}(s, a)$, the state reached by taking a in s ;
- the *goal states* $G \subseteq \mathcal{S}$, tested by the *goal test* $s \in G$;
- the *step costs* $c(s, a, s') > 0$ of taking a in s to reach s' .

Together, s_1 , actions and result define the *state space*, the set of states reachable from s_1 by any sequence of actions. A *solution* is an action sequence $a_{1:T}$ that leads from s_1 to a goal state, *i.e.* such that $s_{t+1} = \text{result}(s_t, a_t)$ for $t = 1, \dots, T$ and $s_{T+1} \in G$. Its *path cost* is

$$\sum_{t=1}^T c(s_t, a_t, s_{t+1}),$$

and an *optimal solution* is a solution of minimal path cost.

Search algorithms

TREE-SEARCH expands the nodes n of the *fringe*, each a path from s_1 to a state $s(n)$, until a goal state leaves it. GRAPH-SEARCH also keeps the set of expanded states, the *closed* set, which avoids repeated states and cycles.

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure
  fringe  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node  $\leftarrow$  REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    fringe  $\leftarrow$  INSERTALL(EXPAND(node, problem), fringe)
```

```
function GRAPH-SEARCH(problem, fringe) returns a solution, or failure
  closed  $\leftarrow$  an empty set
  fringe  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node  $\leftarrow$  REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    if STATE[node] is not in closed then
      add STATE[node] to closed
      fringe  $\leftarrow$  INSERTALL(EXPAND(node, problem), fringe)
  end
```

Search strategies

A *strategy* is the order in which nodes leave the fringe, based on the path cost $g(n)$ from s_1 to n , a heuristic $h(n)$, or $f(n) = g(n) + h(n)$, the estimated cost of the cheapest solution through n . In the table below, b is the maximum branching factor, d the depth of the least-cost solution, m the maximum path length (possibly infinite), C^* the optimal path cost and $\epsilon > 0$ a lower bound on the step costs.

Strategy	Fringe	Complete	Optimal	Time	Space
Depth-first (DFS)	LIFO stack	if no cycles	no	$O(b^m)$	$O(bm)$
Breadth-first (BFS)	FIFO queue	yes	if c uniform	$O(b^d)$	$O(b^d)$
Iterative deepening	DFS, depth $1, 2, \dots$	yes	if c uniform	$O(b^d)$	$O(bd)$
Uniform-cost (UCS)	priority $g(n)$	if $c \geq \epsilon$	yes	$O(b^{C^*/\epsilon})$	$O(b^{C^*/\epsilon})$
Greedy	priority $h(n)$	if no cycles	no	$O(b^m)$	$O(b^m)$
A*	priority $f(n)$	yes	h admissible (tree)	$O(b^m)$	$O(b^m)$

Heuristics

A *heuristic* $h(n) \geq 0$ estimates the true cost $h^*(n)$ of the cheapest path from n to a goal state. It is zero at goal states, and is often the exact cost of a *relaxed problem*, obtained by removing constraints from the original one.

- *Admissible*: $h(n) \leq h^*(n)$ for all n , i.e. h never overestimates. A* with TREE-SEARCH is then optimal.
- *Consistent*: $h(n) \leq c(n, a, n') + h(n')$ for every node n and every successor n' generated by an action a . A consistent heuristic is admissible, f never decreases along a path, and A* with GRAPH-SEARCH is then optimal.
- *Dominance*: if h_1 and h_2 are admissible and $h_2(n) \geq h_1(n)$ for all n , then h_2 dominates h_1 : it is closer to h^* , and A* expands no more nodes with it. The maximum $\max(h_1, h_2)$ of two admissible heuristics is admissible and dominates both.

In session: exercises 1, 3 (a)–(c), 5.

Exercise 1 Path finding

Tram'Ardent maintains the green areas along the Liège tram line with a new robot. This robot's goal is to convey the grass cuttings left by the gardeners to a container as fast as possible. Assuming that

- the fuel cost used by the robot is negligible
- the robot can only navigate on guides that are placed in a rectangular grid configuration and can locate itself on the grid
- there is only one pile and one container by zone (*i.e.* by grid) for which positions are known by the robot
- the speed of the robot is constant and the grass loading/unloading is instantaneous
- the robot can only carry a limited weight of grass at once
- the robot can know the initial weight of grass on the pile.

For a given grid

- (a) Formulate a corresponding search problem.

Solution. States: On an $H \times W$ grid, let a state be a tuple $s = (i, j, w_G, L)$, where $(i, j) \in \{1, \dots, W\} \times \{1, \dots, H\}$ is the position of the robot, $w_G \geq 0$ the weight of grass left on the pile and $L \in \{\text{True}, \text{False}\}$ whether the robot is loaded. The positions $P = (i_p, j_p)$ of the pile and $C = (i_c, j_c)$ of the container, and the weight limit $w_{\max}$ of the robot, are fixed and known, so they are not part of the state.

Initial state: No specific information is given, we can assume it starts anywhere on the grid, unloaded. $s_1 = (i_0, j_0, w_{G0}, \text{False})$

Actions: $a \in \{\text{LEFT}, \text{UP}, \text{RIGHT}, \text{DOWN}\}$

Transition model: $\text{result}(s, a) = (i', j', w'_G, L')$, with

$$(i', j') = \begin{cases} (i + 1, j) & \text{if } a = \text{RIGHT} \text{ and } i < W \\ (i, j + 1) & \text{if } a = \text{UP} \text{ and } j < H \\ (i - 1, j) & \text{if } a = \text{LEFT} \text{ and } i > 1 \\ (i, j - 1) & \text{if } a = \text{DOWN} \text{ and } j > 1 \\ (i, j) & \text{otherwise} \end{cases}$$

$$w'_G = \begin{cases} w_G - \min(w_G, w_{\max}) & \text{if } \neg L \text{ and } (i', j') = P \\ w_G & \text{otherwise} \end{cases}$$

$$L' = \begin{cases} \text{True} & \text{if } L \text{ and } (i', j') \neq C \\ \text{True} & \text{if } w_G > 0 \text{ and } (i', j') = P \\ \text{False} & \text{otherwise} \end{cases}$$

Goal states: $G = \{(i, j, w_G, L) \in \mathcal{S} : \neg L \text{ and } w_G = 0\}$.

Step cost: Under the given assumptions, every step from one state to the next is equivalent. Then $c(s, a, s') = 1$ is relevant. Note that it could be any constant step cost.

- (b) The terrain is rugged at some places, where the robot slows down: a step into a rugged cell costs more than 1, the cost of a step on flat terrain. Propose an admissible heuristic, which estimates the remaining cost at any node.

Solution. Consider the same problem on flat terrain, where every step costs 1. Its optimal cost never exceeds the true remaining cost, since real steps cost at least 1, so it is an admissible heuristic.

On flat terrain, the robot only walks between its position $X = (i, j)$, the pile P and the container C , along shortest paths of length $d(X, Y) = |i_X - i_Y| + |j_X - j_Y|$ on the grid. For a node n in state $s(n) = (i, j, w_G, L)$, $k = \lceil w_G / w_{\max} \rceil$ loads are left on the pile. A loaded robot carries its load to C , then makes k round trips from C to P and back. An unloaded robot walks to P , then makes k trips from P to C and $k - 1$ trips back. Hence

$$h(n) = \begin{cases} 0 & \text{if } \neg L \text{ and } w_G = 0 \\ d(X, C) + 2k d(P, C) & \text{if } L \\ d(X, P) + (2k - 1) d(P, C) & \text{if } \neg L \text{ and } w_G > 0. \end{cases}$$

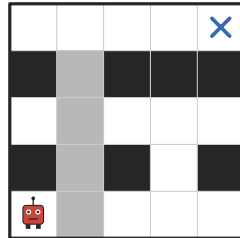
- (c) The robot can now also move diagonally, a diagonal step costing $\sqrt{2}$. Is your heuristic still admissible? If not, what should you change? What if the robot is twice as fast diagonally, so that a diagonal step costs $\sqrt{2}/2$?

Solution. No. If the container is one diagonal step away from the loaded robot, *e.g.* at $(i + 1, j + 1)$, the robot reaches it at a cost of $\sqrt{2}$, while the Manhattan distance counts $d(X, C) = 2$: h overestimates. Taking for d the Euclidean distance makes it admissible again, since every step, straight or diagonal, costs at least the length it covers. Following (b), the exact cost of a path on flat terrain is tighter: with $\Delta_i = |i_X - i_Y|$ and $\Delta_j = |j_X - j_Y|$, the octile distance $d(X, Y) = \max(\Delta_i, \Delta_j) + (\sqrt{2} - 1) \min(\Delta_i, \Delta_j)$ is admissible and dominates the Euclidean distance.

If a diagonal step costs $\sqrt{2}/2$, half the length $\sqrt{2}$ it covers, a unit of length costs at least $1/2$, and d can be the Euclidean distance divided by 2. The bound is reached on a perfect diagonal, such as a path from P to C along a diagonal of the grid.

Exercise 2 Sewer inspection (CS188, Spring 2019)

You control an inspection robot in a rectangular maze of sewers of size $M \times N$, as shown in the figure below. At each time-step, the robot can move into a free adjacent cell or stay in its current location. All actions have a unit cost.



In this particular case, the robot must pass through a series of partially flooded tunnels, as illustrated by the gray cells on the map. In a flooded cell, the robot runs on a sealed backup battery, which lasts A time-steps in a row. Moving into a flooded cell uses 1 unit of that battery, while moving into a free cell recharges it completely.

- (a) Give a minimal state space for this problem (do not include extra information). You should answer for a general instance of the problem, not the specific map above.

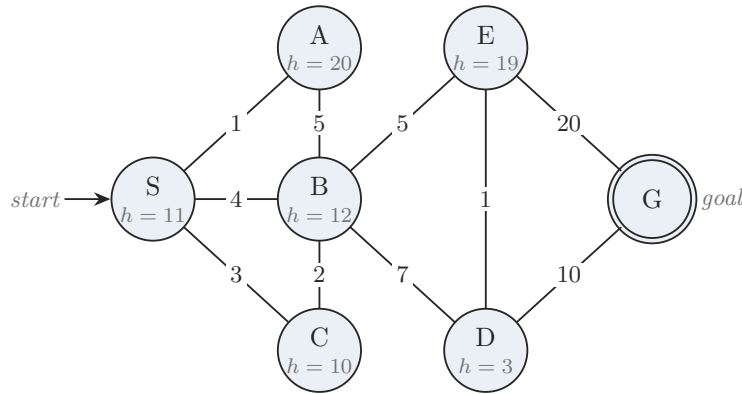
Solution. The position of the robot as well as the number of units of battery it has left, $s = (x, y, u_b) \in \{1, \dots, M\} \times \{1, \dots, N\} \times \{0, \dots, A\}$. The battery holds A units in a free cell and drops to 0 after A moves through flooded cells.

- (b) Give the size of your state space.

Solution. $M \times N \times (A + 1)$, since the battery left takes the $A + 1$ values $0, 1, \dots, A$.

Exercise 3 Route planning

A navigation app plans a route from junction S to junction G on the road network drawn below, whose edges are labelled with travel times in minutes. The heuristic value h of each junction, on its vertex, is the travel time to G predicted by a learned model. For each of the following search algorithms, apply the GRAPH-SEARCH algorithm of the recap to this undirected graph, and give the order in which the states are expanded, as well as the final path returned by the algorithm. Ties are broken by alphabetical order on the states and, between two nodes of the same state, in favour of the one inserted first in the fringe.



- (a) Depth-first search (DFS)

Solution. We use a Last-In First-Out (LIFO) fringe, *i.e.* a *stack*, to perform the search. In this stack, we keep track of the states that are reachable from the closed states. These states are annotated with the state they were expanded from, which allows to reconstruct the path at the end. At each step, the state at the *top* of the stack is *popped*. If its state is not closed yet, it is expanded, which consists in *pushing* its successors to the *top* of the stack.

In the tables below, each row corresponds to one node leaving the fringe, the next one to leave being the rightmost. For readability, a node whose state is already closed is not written down, since it would be discarded when it leaves the fringe; a node whose state gets closed while it waits is discarded in the same way.

Fringe (LIFO stack)	Closed
S	
C(S) B(S) A(S)	S
C(S) B(S) B(A)	S A
C(S) B(S) E(B) D(B) C(B)	S A B
C(S) B(S) E(B) D(B)	S A B C
C(S) B(S) E(B) G(D) E(D)	S A B C D
C(S) B(S) E(B) G(D) G(E)	S A B C D E

Expansion: S, A, B, C, D, E. Path: S, A, B, D, E, G.

(b) Breadth-first search (BFS)

Solution. We use a First-In First-Out (FIFO) fringe, *i.e.* a *queue*, to keep track of the reachable states. At each step, the state at the *front* of the queue is popped. If its state is not closed yet, it is expanded, which consists in pushing its successors to the *back* of the queue.

Fringe (FIFO queue)	Closed
S	
C(S) B(S) A(S)	S
B(A) C(S) B(S)	S A
E(B) D(B) C(B) B(A) C(S)	S A B
E(B) D(B) C(B) B(A)	S A B C
E(B) D(B) C(B)	S A B C
E(B) D(B)	S A B C
G(D) E(D) E(B)	S A B C D
G(E) G(D) E(D)	S A B C D E
G(E) G(D)	S A B C D E

Expansion: S, A, B, C, D, E. Path: S, B, D, G.

(c) Uniform-cost search (UCS)

Solution. The fringe is a *priority* queue. The priority of a node n in the queue is its cumulative cost $g(n)$.

Fringe (priority queue)	Closed
S(0)	
B(S, 4) C(S, 3) A(S, 1)	S
B(A, 6) B(S, 4) C(S, 3)	S A
B(A, 6) B(C, 5) B(S, 4)	S A C
D(B, 11) E(B, 9) B(A, 6) B(C, 5)	S A C B
D(B, 11) E(B, 9) B(A, 6)	S A C B
D(B, 11) E(B, 9)	S A C B
G(E, 29) D(B, 11) D(E, 10)	S A C B E
G(E, 29) G(D, 20) D(B, 11)	S A C B E D
G(E, 29) G(D, 20)	S A C B E D

Expansion: S, A, C, B, E, D. Path: S, B, E, D, G.

(d) Greedy search

Solution. The fringe is also a *priority* queue, but the priority of a node n in the queue is its *heuristic* cost $h(n)$.

Fringe (priority queue)	Closed
S(11)	
A(S, 20) B(S, 12) C(S, 10)	S
A(S, 20) B(C, 12) B(S, 12)	S C
A(B, 20) A(S, 20) E(B, 19) B(C, 12) D(B, 3)	S C B
A(B, 20) A(S, 20) E(D, 19) E(B, 19) B(C, 12) G(D, 0)	S C B D

Expansion: S, C, B, D. Path: S, B, D, G.

(e) A*

Solution. The priority of a node n is the *estimated* cost $f(n) = g(n) + h(n)$ of the cheapest solution through n .

- Is the heuristic admissible? If not, make it admissible.

Solution. A heuristic h is admissible if $0 \leq h(n) \leq h^*(n)$, where $h^*(n)$ is the true forward cost of n . In the graph, $h(E) = 19$ is larger than $h^*(E) = 11$: the model

overestimates the travel time from E, making h non-admissible. This costs the optimal path: D is expanded before E and A* returns S, B, D, G, of cost 21, instead of S, B, E, D, G, of cost 20.

Setting $h(E) = 11$ is sufficient to make h admissible.

- ii. Is the heuristic consistent? If not, can we apply GRAPH-SEARCH?

Solution. A heuristic h is consistent if $h(n) \leq c(n, a, n') + h(n')$ for every node n and every successor n' generated by an action a . Because this graph is undirected, consistency requires $c(n, a, n') \geq |h(n) - h(n')|$ for every pair of neighbors, which does not hold here: $h(A) = 20 > c(A, a, S) + h(S) = 1 + 11$. Correcting $h(E)$ to 11 does not help, as this edge still violates the condition.

This is problematic as it could create situations where a node has been expanded within a path that is not optimal, and cannot be re-expanded later on within the optimal one. Hence, the algorithm loses its optimality.

Fortunately, there is a modification of GRAPH-SEARCH that can handle inconsistent heuristics. If we annotate the nodes in the closed set with the priority at which they were expanded, we can re-expand a node if it has a better (lower) priority than before. The priority in the closed set is then updated.

- iii. Apply A* with the admissible heuristic, using the variant of GRAPH-SEARCH that re-expands a closed state reached with a lower priority.

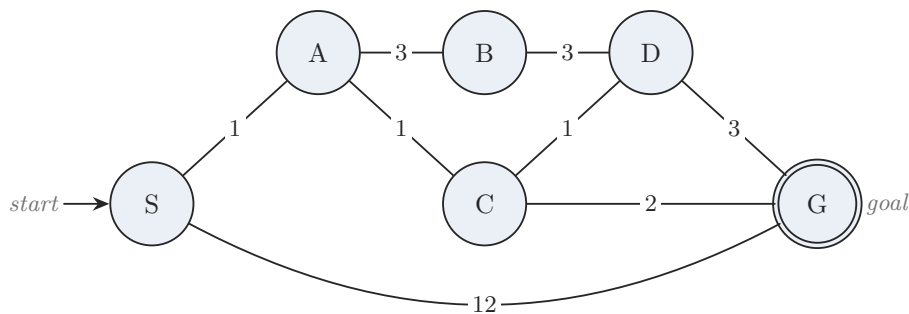
Solution. For readability, nodes that are already in the closed set with a lower priority are not added to the fringe. The re-expansion is shown in red.

Fringe (priority queue)	Closed
S(11)	
A(S, 21) B(S, 16) C(S, 13)	S(11)
A(S, 21) B(C, 17) B(S, 16)	S(11) C(13)
A(B, 29) A(S, 21) E(B, 20) B(C, 17) D(B, 14)	S(11) C(13) B(16)
A(B, 29) E(D, 23) G(D, 21) A(S, 21) E(B, 20) B(C, 17)	S(11) C(13) B(16) D(14)
A(B, 29) E(D, 23) G(D, 21) A(S, 21) E(B, 20)	S(11) C(13) B(16) D(14)
G(E, 29) A(B, 29) E(D, 23) G(D, 21) A(S, 21) D(E, 13)	S(11) C(13) B(16) D(14) E(20)
G(E, 29) A(B, 29) E(D, 23) G(D, 21) A(S, 21) G(D, 20)	S(11) C(13) B(16) D(13) E(20)

Expansion: S, C, B, D, E, D. Path: S, B, E, D, G.

Exercise 4 Pathfinding in a video game

A character of a video game, controlled by the game, moves from S to G between the waypoints of a level, drawn below with the lengths of the passages between them; the direct passage from S to G is a long one. Apply the GRAPH-SEARCH algorithm of the recap to this graph, with the tie-breaking of exercise 3.



- (a) Apply iterative deepening, with a depth limit growing from 1. In which order are the states expanded, which path is returned, and is it optimal?

Solution. With a depth limit of 1, S is expanded and generates A and G at depth 1. A leaves the fringe first (alphabetical order), is not a goal and cannot be expanded at the depth limit; G leaves the fringe next and passes the goal test.

Expansion: S. Path: S, G, of cost 12. It is not optimal, the cheapest path being S, A, C, G, of cost 4. Like breadth-first search, iterative deepening returns the shallowest goal, which is the cheapest one only when the path cost is non-decreasing with the depth.

- (b) The designers of the level propose two estimates of the distance to G:

State	h_1	h_2
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0

which one is not admissible? Why?

Solution. The heuristic h_1 is not admissible because the real cost from S to G is 4 which is smaller than 5.

- (c) Is the admissible heuristic consistent? Does it matter here?

Solution. No: $h_2(S) = 4 > c(S, a, A) + h_2(A) = 1 + 2$, so h_2 is admissible but not consistent. It does not matter on this graph, where no state is expanded before its cheapest path is found, so that graph-search A* still returns the optimal path. Consistency is what guarantees this in general.

- (d) Apply A* with the admissible heuristic.

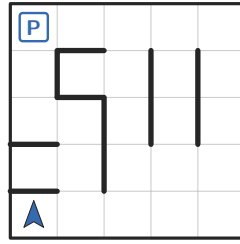
Solution. Expansion: S, A, C. Path: S, A, C, G.

- (e) Which of these strategies are guaranteed to return the optimal path on this graph?

Solution. Uniform-cost search, and A* with an admissible heuristic in tree search or a consistent one in graph search, since they order the fringe by cost. Depth-first search and greedy search give no guarantee. Breadth-first search and iterative deepening return the shallowest goal, S, G here, which is not the cheapest one because the step costs differ.

Exercise 5 Self-parking car (CS188, Spring 2014)

A self-parking car must reach a free parking spot in a car park, seen from above as a grid with walls. At all time-steps, the car points at direction $d \in \{N, S, E, W\}$. With a single action, the car can either move forward at an adjustable velocity $v \in [0, V]$ or turn. The turning actions are **left** and **right**, which change the car's direction by 90 degrees. Turning is only permitted when the velocity is zero (and leaves it at zero). The moving actions are **faster** and **slower**. The action **faster** increments the velocity by 1 and **slower** decrements the velocity by 1; in both cases the car then moves a number of squares equal to its *new* velocity. There is no action that keeps the velocity: the car cannot move at the same nonzero velocity for two consecutive time steps. Any action whose move would cross a wall is illegal, whichever square the car stops on. Any action that would take v below 0 or above a maximum speed V is also illegal. A car moving too fast towards a wall may therefore be left with no legal action. The goal is a plan which parks the car (stationary) on the free spot using as few actions (time steps) as possible.



As an example, in the car park above, if the car is initially stationary, it might first turn to the east using **right**, then move one square east using **faster**, then two more squares east using **faster** again. However, the car will have to slow to take the turn.

- (a) For a grid of size $M \times N$, what is the size of the state space? Assume that all configurations are reachable from the starting state.

Solution. $N \times M \times 4 \times (V + 1)$

- (b) What is the maximum branching factor of this problem? Assume that illegal actions are simply not in actions(s).

Solution. The maximum branching factor is 3, and this happens when the car is stationary. While stationary, it can take the following 3 actions: **faster**, **left**, **right**.

- (c) Is the Manhattan distance from the car's location to the free spot admissible?

Solution. No because it does not take the velocity of the car into account. For instance, in a long straight path, the number of steps could be smaller than the number of squares.

- (d) If we used an inadmissible heuristic in A* tree search, could it change the completeness of the search? And the optimality?

Solution. With an inadmissible heuristic, A* will still find a final state if there exists a reachable one (complete), but it could stop the search before finding the optimal one (non optimal).

- (e) State and motivate a non-trivial admissible heuristic for this problem.

Solution. Some examples of admissible heuristics are:

- The Manhattan distance to the free spot divided by the maximum speed V , since the car covers at most V squares per action.
- The current velocity v , since the car needs v actions of **slower** to park on the free spot.
- The maximum of the two, $\max\{\frac{\text{Manhattan}}{V}, v\}$, which *dominates* both and is still admissible, as both are lower bounds on the true forward cost. Their *sum* is not admissible: one square away from the free spot at velocity $v = 2$, parking takes 2 actions (**slower** twice, which moves the car by 1 then 0 squares), while the sum gives $1/V + 2 > 2$.

- (f) Give a general advantage that an inadmissible heuristic might have over an admissible one.

Solution. The time to solve an A* tree search problem is a function of two factors: the number of nodes expanded, and the time spent per node.

- An inadmissible heuristic may be faster to compute, leading to a solution that is obtained faster due to less time spent per node.
- Admissible heuristics are sometimes too conservative (well below h^*). An inadmissible heuristic could be a better estimate of the true forward cost h^* (although it will overestimate at times), thus expanding fewer nodes.

We lose the guarantee of optimality by using an inadmissible heuristic. However, sometimes, we may be okay with finding a suboptimal solution faster.

Exercise 6 Learned heuristic (CS188, Spring 2019)

Heuristics are increasingly learned: a neural network, trained on solved instances of a problem, predicts the remaining cost of a node. Consider a graph search problem where all edges have a unit cost and the optimal solution has cost C^* , and a network calibrated to never overestimate, which predicts $h(n) = \max\{h^*(n) - k, 0\}$, where $h^*(n)$ is the true forward cost of n and $k \leq C^*$ is a non-negative constant.

(a) Is h admissible?

Solution. Yes. If $h^* \geq k$, $h^* - k \leq h^*$, otherwise $0 \leq h^*$.

(b) Which of the following is the most reasonable description of how much more work will be done, *i.e.* how many more nodes will be expanded, with heuristic h compared to h^* , as a function of k ?

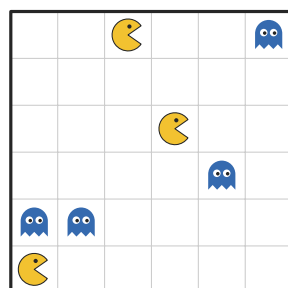
- ☐ Constant in k
- ☐ Linear in k
- ☒ Exponential in k
- ☐ Unbounded

Solution. A^* expands a node n when $f(n) = g(n) + h(n) \leq C^*$. With $h(n) = \max\{h^*(n) - k, 0\}$, this happens as soon as $g(n) + h^*(n) \leq C^* + k$, *i.e.* as soon as the cheapest path through n costs at most k more than the optimal one. Since all edges have a unit cost, such a path is at most k steps longer than the optimal path, and there are $O(b^k)$ of them in the worst case, for a branching factor b . The additional work is therefore exponential in k .

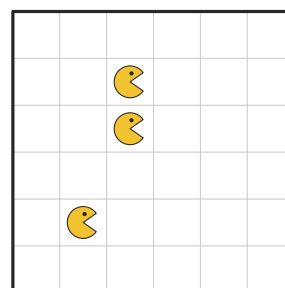
Exercise 7 Pacmen (Exam, January 2023)

Pacman and his friends have decided to combine forces and go on the offensive, and are now chasing ghosts! In a grid of size $M \times N$, Pacman and $P - 1$ of his friends are moving around to collectively eliminate all of the ghosts in the grid by stepping on the same square as each of them. Moving into the same square as a ghost will eliminate it from the grid. At every turn, Pacman and his friends may choose one of the four (**north**, **south**, **east**, **west**) actions but may not collide with each other. In other words, any action that would result in two or more Pacmen occupying the same square will result in no movement for either of the Pacmen. Additionally, Pacman and his friends are *indistinguishable* from each other. There are a total of G ghosts, which are also indistinguishable from each other and cannot move.

Treating this as a search problem, we consider each configuration of the grid to be a state, and the goal state to be the configuration where all ghosts have been eliminated from the board. The cost of each turn (all Pacmen move once) is 1. Below is an example starting state, as well as an example of goal state.



Initial state example



Final state example

Suppose that Pacman has no friends ($P = 1$).

- (a) What is the size of the minimal state space representation? Explain your answer.

Solution. The state s is fully described with the position of Pacman and a boolean for each ghost (eaten or not). The size of the state space is therefore $M \times N \times 2^G$. We don't need to save the position of the ghosts within the state because they never change.

- (b) Explain whether each of the following heuristics is admissible, consistent, neither, or both.

- $h_1(n)$ = the sum of the Manhattan distances from Pacman to every remaining ghost.
- $h_2(n)$ = the number of ghosts times the maximum Manhattan distance between Pacman and any of the ghosts.
- $h_3(n)$ = the number of remaining ghosts.

Solution. A heuristic h is admissible if $0 \leq h(n) \leq h^*(n)$, where $h^*(n)$ is the true forward cost of n . It is consistent if $h(n) \leq c(n, a, n') + h(n')$ for every successor n' generated by an action a . Note that a non-admissible heuristic cannot be consistent.

In the node n below, $h_1(n) = 2 + 3 + 4 = 9$ and $h_2(n) = 3 \times 4 = 12$ are larger than $h^*(n) = 4$ (not admissible). Moving Pacman one cell to the right leads to n' , for which $h_1(n') = 6$ and $h_2(n') = 3 \times 3 = 9$, so that $h_1(n) > c(n, a, n') + h_1(n') = 7$ and $h_2(n) > c(n, a, n') + h_2(n') = 10$ (not consistent).



Conversely, for any node n , $h_3(n)$ is a lower bound on the number of remaining turns (admissible) and a single action removes at most one ghost, so that $h_3(n) \leq c(n, a, n') + h_3(n')$ with $c(n, a, n') = 1$ (consistent).

Suppose that Pacman has exactly one less friend than the number of ghosts ($P = G$).

- (c) What is the size of the minimal state space representation? Explain your answer.

Solution. The Pacmen are indistinguishable and occupy distinct squares, so a state is a set of P positions together with a boolean for each ghost: $\binom{MN}{P} \times 2^G$ states. Both $(M \times N)^P \times 2^G$, which distinguishes the Pacmen, and $2^{M \times N} \times 2^G$, which represents the Pacmen by one boolean per square and thus allows any number of them, are larger.

- (d) Explain whether each of the following heuristics is admissible, consistent, neither, or both.

- $h_4(n)$ = the largest of the Manhattan distances between each Pacman and its closest ghost.
- $h_5(n)$ = the smallest of the Manhattan distances between each Pacman and its closest ghost.
- $h_6(n)$ = the number of remaining ghosts.
- $h_7(n)$ = the number of remaining ghosts divided by P .

Solution. Since P Pacmen move at every turn, up to P ghosts disappear per turn, which breaks the two heuristics that count turns as if a single Pacman were chasing.

- h_4 is not admissible, hence not consistent. Put two ghosts next to the same Pacman and the remaining Pacmen far away: two turns are enough ($h^* = 2$), while h_4 is the distance of the farthest Pacman to its closest ghost, which can be arbitrarily large.
- h_6 is not admissible either, hence not consistent. If every ghost is adjacent to a different Pacman, one turn ends the game ($h^* = 1$) while $h_6 = G > 1$.
- h_5 is admissible, as some Pacman must reach the ghost closest to it, which takes at least $h_5(n)$ turns, and consistent, as a turn changes each distance by at most 1.
- h_7 is admissible, as at least G/P turns are needed to eliminate G ghosts, and consistent, as a turn decreases the number of remaining ghosts by at most P .

1. An agent is rational if ...
 - ☐ its actions always lead to the best possible outcome.
 - ☐ it knows the state s_t of the environment at all times.
 - ☒ its policy maximizes $\mathbb{E}[V(s_{1:T}) \mid \pi]$, given its percepts and prior knowledge.
 - ☐ it maximizes the performance measure that its designer had in mind.
2. As a task environment, poker is ...
 - ☐ discrete, multi-agent and deterministic.
 - ☒ partially observable, stochastic and multi-agent.
 - ☐ partially observable, episodic and single-agent.
 - ☐ None of the above.
3. In a stochastic environment, a simple reflex agent ($a_t = \pi(e_t)$) given the same percept twice will ...
 - ☒ take the same action every time.
 - ☐ take a different action every time.
 - ☐ take a different action most of the time.
 - ☐ take the same action most of the time.
4. A goal-based agent ...
 - ☐ selects its action from the current percept e_t only.
 - ☒ searches its transition model for an action sequence that reaches a state of G .
 - ☐ prefers a fast and safe route over a slow and risky one.
 - ☐ maximizes the expected performance $\mathbb{E}[V(s_{1:T}) \mid e_{1:t}, a_t]$ of its actions.
5. The A* algorithm, in its graph version (GRAPH-SEARCH, without re-expansion) is ...
 - ☐ always optimal.
 - ☐ always optimal if the heuristic is admissible.
 - ☒ always optimal if the heuristic is consistent.
 - ☐ never optimal when there are cycles.
6. Which of the following statements about heuristics is true?
 - ☐ $h(n) = 0 \forall n$ can speed up A* compared to UCS.
 - ☐ If a heuristic is admissible, it is guaranteed to be consistent.
 - ☐ The average step cost $h(n) = \frac{1}{|\text{actions}(s)|} \sum_a c(s, a, \text{result}(s, a))$, with $s = s(n)$, is admissible.
 - ☒ The true remaining cost dominates all admissible heuristics.
7. Iterative deepening is often preferred to breadth-first search because it ...
 - ☐ expands strictly fewer nodes.
 - ☒ finds a solution at the same depth d with $O(bd)$ memory, instead of $O(b^d)$.
 - ☐ is optimal even when the step costs differ.
 - ☐ never expands the same state twice.
8. In GRAPH-SEARCH, keeping a closed set ...
 - ☒ makes depth-first search complete on a finite state space.
 - ☐ makes any admissible heuristic consistent.
 - ☐ guarantees that A* is optimal whenever h is admissible.
 - ☐ is only useful when the state space is infinite.